VNiVERSiDAD D SALAMANCA

Creating Electronic Books with Code and Artificial Intelligence Assistants

Course for the Faculties of Sciences and Chemical Sciences

Álvaro Lozano Murciego and André Sales Mendes

Universidad de Salamanca

1st Edition • 2026

CONTENTS

1 Session 0. Ubuntu Terminal for Verilog **3**
- 1.1 The terminal as a workbench . . . 3
- 1.2 Where am I: `pwd` . . . 3
- 1.3 What is here: `ls` . . . 4
- 1.4 Change folder: `cd` . . . 4
- 1.5 Create and prepare a practice folder . . . 4
- 1.6 Check installed tools . . . 5
- 1.7 Create a first text file . . . 5
- 1.8 Compile and run . . . 6
- 1.9 Work pattern for the sessions . . . 6
- 1.10 Basic help commands . . . 7
- 1.11 Common mistakes . . . 7
- 1.12 Guided practice . . . 7
- 1.13 Closing checkpoint . . . 8

2 Session 1. Introduction to Verilog **9**
- 2.1 Key concepts . . . 9
- 2.2 Guided practice . . . 9
- 2.3 Theory-practice-figures index . . . 10
- 2.4 Reference figures . . . 10
- 2.5 Closing checkpoint . . . 11
- 2.6 Original source . . . 11

3 Session 2. Bit Operations **13**
- 3.1 Key concepts . . . 13
- 3.2 Guided practice . . . 13
- 3.3 Theory-practice-figures index . . . 14
- 3.4 Work without an associated figure . . . 14
- 3.5 Closing checkpoint . . . 14
- 3.6 Original source . . . 14

4 Session 3. Logic Gates **15**
- 4.1 Key concepts . . . 15
- 4.2 Guided practice . . . 15
- 4.3 Theory-practice-figures index . . . 16
- 4.4 Reference figures . . . 16
- 4.5 Closing checkpoint . . . 20
- 4.6 Original source . . . 20

5 Session 4. Modules **23**

5.1 Key concepts . . . 23
5.2 Guided practice . . . 23
5.3 Theory-practice-figures index . . . 24
5.4 Reference figures . . . 24
5.5 Closing checkpoint . . . 27
5.6 Original source . . . 27

6 Session 5. Routers and Adders **29**
6.1 Key concepts . . . 29
6.2 Guided practice . . . 29
6.3 Theory-practice-figures index . . . 30
6.4 Reference figures . . . 30
6.5 Closing checkpoint . . . 34
6.6 Original source . . . 34

7 Session 6. Flip-Flops **37**
7.1 Key concepts . . . 37
7.2 Guided practice . . . 37
7.3 Theory-practice-figures index . . . 38
7.4 Reference figures . . . 38
7.5 Closing checkpoint . . . 41
7.6 Original source . . . 41

8 Session 7. Registers **43**
8.1 Key concepts . . . 43
8.2 Guided practice . . . 43
8.3 Theory-practice-figures index . . . 44
8.4 Reference figures . . . 44
8.5 Closing checkpoint . . . 46
8.6 Original source . . . 46

9 Session 8. Counters **47**
9.1 Key concepts . . . 47
9.2 Guided practice . . . 47
9.3 Theory-practice-figures index . . . 48
9.4 Reference figures . . . 48
9.5 Closing checkpoint . . . 53
9.6 Original source . . . 53

10 Session 9. ALU **55**
10.1 Key concepts . . . 55
10.2 Guided practice . . . 55
10.3 Theory-practice-figures index . . . 56
10.4 Reference figures . . . 56
10.5 Closing checkpoint . . . 58
10.6 Original source . . . 58

Welcome to the **Computers I** practice book.

This material supports the study of digital design fundamentals using **Verilog** as a hardware description language. The course starts from a simple idea: a computer is not a magic box, but an ordered composition of signals, logic gates, modules, registers, counters, and arithmetic-logic units.

What you will learn

By working through this book, you will learn to:

- describe digital circuits with Verilog modules;
- simulate the behavior of logic gates, buses, flip-flops, registers, and counters;
- interpret signals, states, and propagation delays;
- build combinational and sequential components step by step;
- connect each exercise with the reference diagrams and tables for the session.

How the course is organized

The course is organized into practice sessions. Each session includes a short theoretical explanation, guided exercises, an index connecting theory, practice, and figures, and reference diagrams to keep the circuit being programmed clearly in view.

Recommended workflow

Before writing code, identify the circuit inputs, outputs, and internal signals. Then consult the reference figures, write the Verilog module, and check the simulation. If the result does not match what you expected, return to the figure and review the connections.

Session roadmap

Table1: Initial map of the course

Session	Topic	Main idea
0	Ubuntu Terminal for Verilog	Folders, files, basic editors, and first contact with `iverilog`
1	Introduction to Verilog	Working environment, basic modules, and screen output
2	Bit operations	Masks, parity, and bit reduction
3	Logic gates	AND, OR, NOT, NAND, NOR, XOR, XNOR, and combinational functions
4	Modules	Interfaces, hierarchy, comparators, and encoders
5	Routers and adders	Buses, multiplexers, half adders, and full adders
6	Flip-flops	Elementary memory, clock, edges, and asynchronous inputs
7	Registers	SISO/SIPO registers and waveform-based debugging
8	Counters	Up/down counting and state transitions
9	ALU	File inclusion and use of the 74181 ALU

Available material

Start with the course preparation session:

- *Session 0. Ubuntu Terminal for Verilog*

You can switch to the Spanish book from the language selector in the top bar.

CHAPTER 1

SESSION 0. UBUNTU TERMINAL FOR VERILOG

This session helps you become comfortable with the terminal. Before writing Verilog circuits, it is useful to know how to move through folders, find files, and run the tools that compile and simulate our designs.

Learning objectives

- Open an Ubuntu terminal and know which folder you are in.
- Navigate the file system with `pwd`, `ls`, and `cd`.
- Create a working folder and organize Verilog files.
- Create a simple file with `gedit` and show it from the terminal.
- Prepare the first compilation command with `iverilog`.

1.1 The terminal as a workbench

A terminal is a window where we type text commands. In this course we will use it as a workbench: enter a folder, inspect its files, open an editor, and run simple commands.

In Ubuntu you can open it with `Ctrl` + `Alt` + `T`, or by searching for **Terminal** in the applications menu.

1.2 Where am I: `pwd`

The command `pwd` prints the path of the current folder.

```
pwd
```

A possible output is:

```
/home/user
```

This path means that we are inside the user's home folder. If the terminal were a file browser, `pwd` would be like looking at the address bar.

1.3 What is here: `ls`

The command `ls` lists the contents of the current folder.

```
ls
```

Useful variants:

```
ls -l
ls -a
ls -lh
```

Command	What it is for
`ls`	Lists visible files and folders.
`ls -l`	Shows more details: permissions, size, and date.
`ls -a`	Includes hidden files, whose names start with a dot.
`ls -lh`	Shows more readable sizes, for example `4K` or `2M`.

1.4 Change folder: `cd`

The command `cd` changes the current folder.

```
cd Documents
pwd
```

To go one folder back:

```
cd ..
```

To go directly to your home folder:

```
cd
```

To enter a specific path:

```
cd ~/verilog/session_00
```

The symbol `~` represents your home folder. In many installations it is equivalent to something like `/home/user`.

1.5 Create and prepare a practice folder

Create one folder for the course and another one for this session.

```
mkdir -p ~/verilog/session_00
cd ~/verilog/session_00
pwd
```

The command `mkdir` creates folders. The option `-p` allows several nested folders to be created if they do not exist yet.

1.6 Check installed tools

To compile Verilog in Ubuntu we will mainly use:

- `iverilog`: compiles the design and the testbench.

Check whether they are available:

```
iverilog -V
```

If Ubuntu prints a version message, the tool is installed. If it prints `command not found`, the tool still needs to be installed.

Installation on Ubuntu

If you are working on a machine where you have permission to install packages, you can prepare the tools with:

```
sudo apt update
sudo apt install iverilog
```

1.7 Create a first text file

Create the file `hello.txt` with `gedit`:

```
gedit hello.txt
```

Other terminal editors

Besides `gedit`, there are editors that open inside the terminal itself, such as `nano`, `pico`, or `vim`. In this session we will use `gedit` because it is a visual and simple starting point.

Write a single word:

```
hello
```

Save the file and close `gedit`.

Check that the file exists:

```
ls -l
```

Show its contents from the terminal:

```
cat hello.txt
```

The expected output is:

```
hello
```

1.8 Compile and run

Later we will write Verilog files. The cycle will be very similar: be in the right folder, list the files, compile, and run the result.

Create this file `greeting.v`:

```
gedit greeting.v
```

Content:

```
module greeting;
  initial begin
    $display("Hello from Verilog");
    $finish;
  end
endmodule
```

Compile the file with `iverilog`:

```
iverilog -o greeting greeting.v
```

This command reads `greeting.v` and creates an executable file named `greeting`.

Run it with:

```
./greeting
```

The expected output is:

```
Hello from Verilog
```

1.9 Work pattern for the sessions

In this course we will repeat the same sequence many times:

```
cd ~/verilog/session_00
ls
iverilog -o simulation file.v
./simulation
```

When the example has a testbench, we will normally compile two files:

```
iverilog -o simulation design.v testbench.v
./simulation
```

1.10 Basic help commands

Command	Typical use
`clear`	Clears the terminal screen.
`history`	Shows previously used commands.
`man ls`	Opens the manual page for `ls`. Exit with `q`.
`cat file.txt`	Shows the contents of a short file.
`cp source.v copy.v`	Copies a file.
`mv old.v new.v`	Renames or moves a file.
`rm file.txt`	Deletes a file. Use it carefully.

1.11 Common mistakes

Message or situation	What to check
`command not found`	The tool is not installed or its name is misspelled.
`No such file or directory`	You are not in the right folder or the file has a different name. Use `pwd` and `ls`.
`cat hello.txt` prints nothing	The file is empty or was not saved in `gedit`.
`./greeting` does not work	Check that you first compiled with `iverilog -o greeting greeting.v`.

1.12 Guided practice

1. Open an Ubuntu terminal.
2. Create the folder `~/verilog/session_00`.
3. Enter that folder and confirm the path with `pwd`.
4. Open `gedit` from the terminal with `gedit hello.txt`.
5. Write `hello`, save the file, and close `gedit`.
6. Use `ls` to check that `hello.txt` appears in the folder.
7. Use `cat hello.txt` to show its contents.
8. Change the text to `hello world`, save again, and show it again with `cat hello.txt`.

1.13 Closing checkpoint

Before moving to session 1, you should be able to answer without looking at notes: where am I (`pwd`), which files do I have (`ls`), how do I enter a folder (`cd`), how do I open a file with `gedit`, and how do I show a short file with `cat`.

CHAPTER 2

SESSION 1. INTRODUCTION TO VERILOG

This session introduces Verilog as a hardware description language and sets up the minimum workflow for compiling and simulating simple examples. The goal is not to write conventional software, but to describe circuits and observe their behavior.

Learning objectives

- Verilog is an HDL: it describes hardware, not just a sequence of software instructions.
- The basic workflow is to write a `.v` file, compile it, and run the simulation.
- Modules delimit the design; an `initial` block defines a sequence of test actions.

2.1 Key concepts

- Verilog is an HDL: it describes hardware, not just a sequence of software instructions.
- The basic workflow is to write a `.v` file, compile it, and run the simulation.
- Modules delimit the design; an `initial` block defines a sequence of test actions.
- Registers (`reg`) store values during simulation and wires (`wire`) connect signals.
- The `$display` format codes help inspect values in binary, octal, decimal, or hexadecimal.

2.2 Guided practice

1. Create a working directory and save a `hello.v` file inside it.
2. Compile the file with `iverilog` and run the generated output.
3. Modify the example so that it prints an integer in decimal, binary, and hexadecimal; compare your output with Fig. 2.1.
4. Declare a 16-bit register and observe what happens when assigning values beyond its capacity; use Fig. 2.2 to identify which code fragment produces each output.

2.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

Theory block	Related exercises	Figures to consult
Output formats and base conversion	Exercises 3 and 4	Fig. 2.1, Fig. 2.2

2.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 2.1 is the reference for: expected output for an introductory conversion and display exercise.

```
module ej_1_9;

integer i;
real f;

initial
begin
  i=4;
  f=2.7172;
  $display("i vale %d y f vale %g",i,f);
end

endmodule
```

Figure2.1: Expected output for an introductory conversion and display exercise.

The Fig. 2.2 is the reference for: commented version of the same example, useful for locating each instruction in the code.

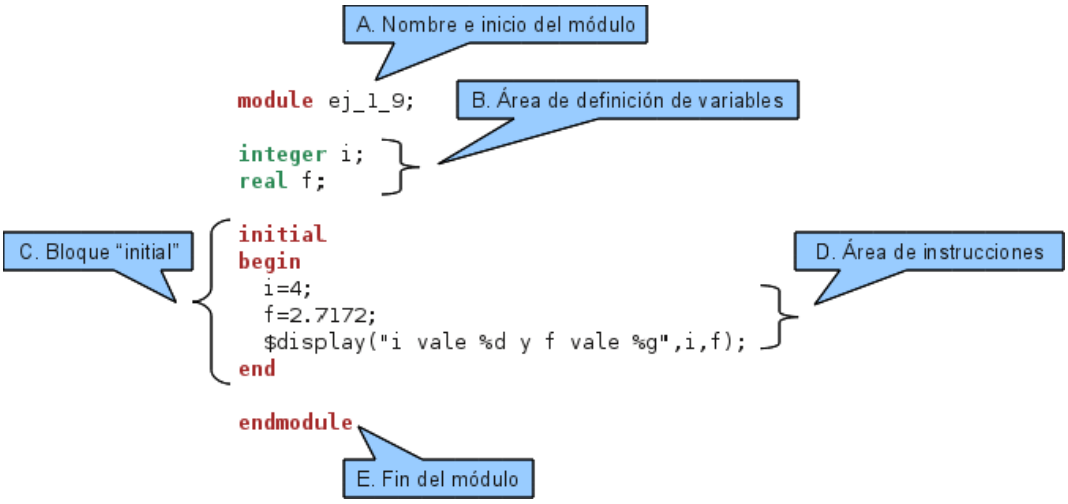


Figure2.2: Commented version of the same example, useful for locating each instruction in the code.

2.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

2.6 Original source

Content translated and expanded from the class presentation and the reference page: http://avellano.fis.usal.es/~compi/sesion1.htm.

CHAPTER 3

SESSION 2. BIT OPERATIONS

This session focuses on bit manipulation with masks. The central idea is to change selected positions of a register while leaving the rest unchanged.

Learning objectives

- One's complement is obtained with ~, turning each 0 into 1 and each 1 into 0.
- To set bits, use a mask with ones in the target positions and a bitwise OR operation.
- To clear bits, combine an inverted mask with a bitwise AND operation.

3.1 Key concepts

- One's complement is obtained with ~, turning each 0 into 1 and each 1 into 0.
- To set bits, use a mask with ones in the target positions and a bitwise OR operation.
- To clear bits, combine an inverted mask with a bitwise AND operation.
- To toggle bits, use bitwise XOR with a mask marking the positions that must change.
- Reduction operators condense many bits into one result, for example to compute parity.

3.2 Guided practice

1. Declare a 16-bit register and assign an initial value that is easy to read in binary.
2. Set one bit, clear three bits, and toggle another bit using masks.
3. Compute the even parity bit of an 8-bit register.
4. Solve the operations by hand first and then compare them with the simulation; in this session the main reference is the bit table you build during the calculation.

3.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

Theory block	Related exercises	Figures to consult
Masks, parity, and bit reduction	Exercises 1 to 4	Manual bit table from the statement; no external figure is associated with this session

3.4 Work without an associated figure

This session relies mainly on operations over registers and masks. Use a handwritten bit table to track each position before running the code.

3.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

3.6 Original source

Content translated and expanded from the class presentation and the reference page: http://avellano.fis.usal.es/~compi/sesion2.htm.

CHAPTER 4

SESSION 3. LOGIC GATES

This session connects the logic gates studied in theory with their instantiation in Verilog. It also introduces the special values `x` and `z`, which are essential in digital simulation.

Learning objectives

- Verilog primitive gates are instantiated by listing the output first and then the inputs.
- The values `x` and `z` represent uncertainty and high impedance.
- A complete truth table must include 0, 1, x, and z when the exercise requires it.

4.1 Key concepts

- Verilog primitive gates are instantiated by listing the output first and then the inputs.
- The values `x` and `z` represent uncertainty and high impedance.
- A complete truth table must include 0, 1, x, and z when the exercise requires it.
- Gate interconnection makes it possible to build more complex combinational functions.
- The same function can be implemented in different ways, for example using only NAND gates.

4.2 Guided practice

1. Complete the AND truth table with the 16 combinations of 0, 1, x, and z; use Fig. 4.1 and Fig. 4.2 as references.
2. Repeat the check for OR, NAND, NOR, XOR, XNOR, and BUFFER; consult Fig. 4.3 through Fig. 4.9 depending on the gate being simulated.
3. Build the proposed function with gates and verify its truth table; use Fig. 4.10, Fig. 4.11, and Fig. 4.12.
4. Reimplement the function using only NAND gates and compare the outputs; follow Fig. 4.14, Fig. 4.13, and Fig. 4.15.

4.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

Theory block	Related exercises	Figures to consult
Basic gates	Exercises 1 and 2	Fig. 4.1, Fig. 4.2, Fig. 4.3, Fig. 4.4, Fig. 4.5, Fig. 4.6, Fig. 4.7, Fig. 4.8, Fig. 4.9
Combinational function f2	Exercise 3	Fig. 4.10, Fig. 4.11, Fig. 4.12
NAND equivalence	Exercise 4	Fig. 4.13, Fig. 4.14, Fig. 4.15

4.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 4.1 is the reference for: basic AND gate.

AND

·	0	1
0	0	0
1	0	1

AND	FALSE	TRUE
FALSE	FALSE	FALSE
TRUE	FALSE	TRUE

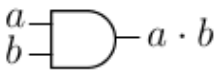


Figure4.1: Basic AND gate.

The Fig. 4.2 is the reference for: instantiation of an AND gate in Verilog.

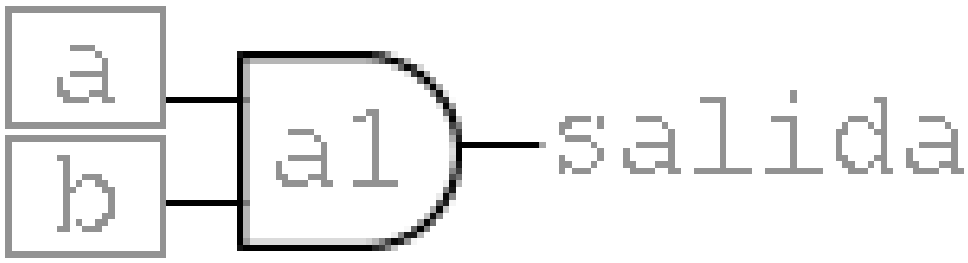


Figure4.2: Instantiation of an AND gate in Verilog.

The Fig. 4.3 is the reference for: oR gate and its conceptual connection.

The Fig. 4.4 is the reference for: nOT gate.

The Fig. 4.5 is the reference for: nAND gate.

The Fig. 4.6 is the reference for: nOR gate.

The Fig. 4.7 is the reference for: xOR gate.

OR

+	0	1
0	0	1
1	1	1

OR	FALSE	TRUE
FALSE	FALSE	TRUE
TRUE	TRUE	TRUE

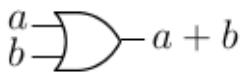


Figure4.3: OR gate and its conceptual connection.

NOT

¯	0	1
	1	0

NOT	FALSE	TRUE
	TRUE	FALSE

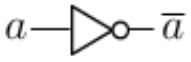


Figure4.4: NOT gate.

NAND

a	b	NAND
0	0	1
0	1	1
1	0	1
1	1	0

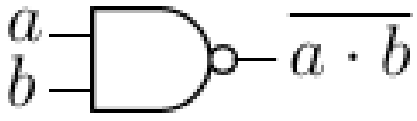


Figure4.5: NAND gate.

NOR

a	b	NOR
0	0	1
0	1	0
1	0	0
1	1	0

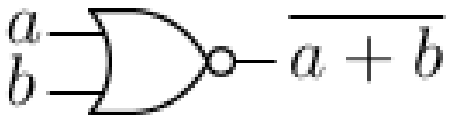


Figure4.6: NOR gate.

XOR

a	b	XOR
0	0	0
0	1	1
1	0	1
1	1	0

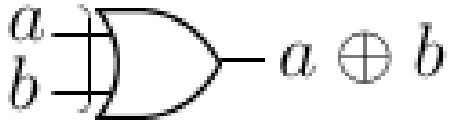


Figure4.7: XOR gate.

The Fig. 4.8 is the reference for: xNOR gate.

XNOR

a	b	XNOR
0	0	1
0	1	0
1	0	0
1	1	1

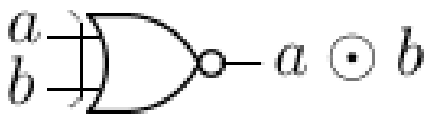


Figure4.8: XNOR gate.

The Fig. 4.9 is the reference for: bUFFER gate.

BUFFER

0	1
0	1

BUFFER	FALSE	TRUE
	FALSE	TRUE

Figure4.9: BUFFER gate.

The Fig. 4.10 is the reference for: combinational logic function f2.

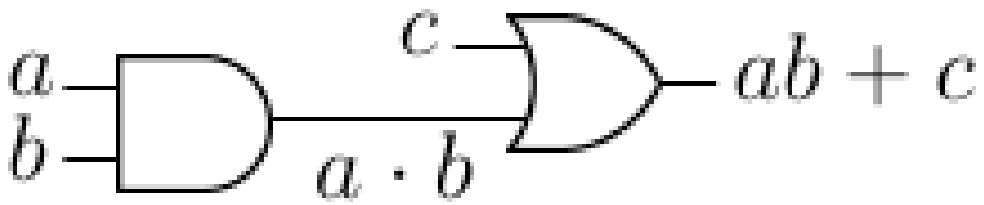


Figure4.10: Combinational logic function f2.

The Fig. 4.11 is the reference for: auxiliary variables for function f2.

The Fig. 4.12 is the reference for: truth table associated with f2.

The Fig. 4.13 is the reference for: algebraic form of function f3.

The Fig. 4.14 is the reference for: gate implementation of f3.

The Fig. 4.15 is the reference for: equivalent implementation using NAND gates.

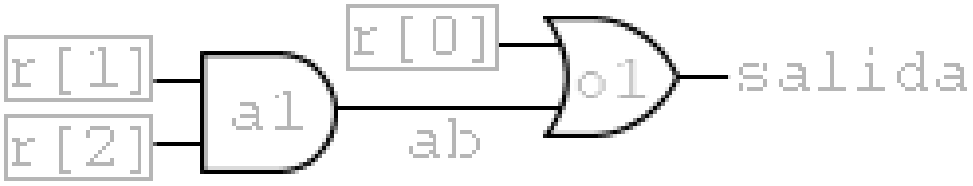


Figure4.11: Auxiliary variables for function f2.

4.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

4.6 Original source

Content translated and expanded from the class presentation and the reference page: http://avellano.fis.usal.es/~compi/sesion3.htm.

a	b	c	ab	f_2
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

Figure4.12: Truth table associated with f2.

$$f_3(a,b,c) = bc + ab\overline{c} + \overline{b}c + c$$

Figure4.13: Algebraic form of function f3.

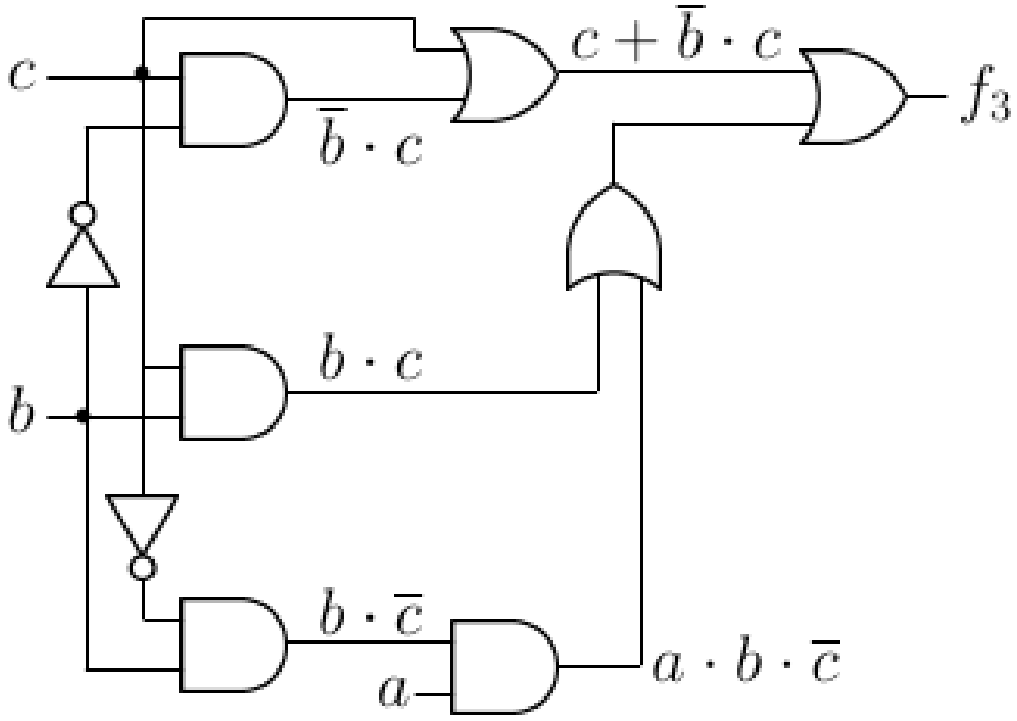


Figure4.14: Gate implementation of f3.

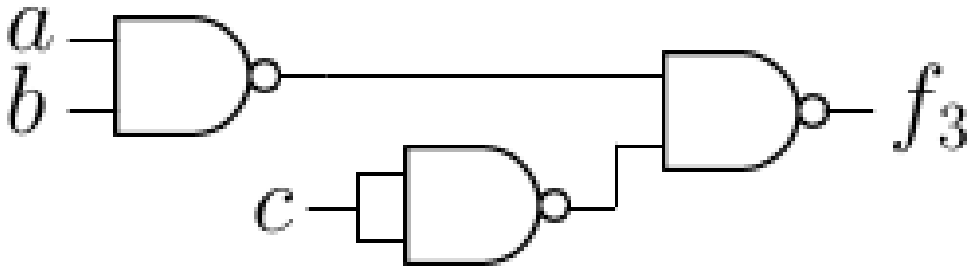


Figure4.15: Equivalent implementation using NAND gates.

CHAPTER 5

SESSION 4. MODULES

This session introduces modular design. A Verilog module encapsulates inputs, outputs, and behavior, like a reusable circuit block.

Learning objectives

- A module needs a clear interface: `input`, `output`, and `inout`.
- Hierarchy lets us build large systems from smaller modules.
- Logic gates can be viewed as predefined modules in the language.

5.1 Key concepts

- A module needs a clear interface: `input`, `output`, and `inout`.
- Hierarchy lets us build large systems from smaller modules.
- Logic gates can be viewed as predefined modules in the language.
- A one-bit comparator is a good first example of a reusable module.
- Encoders show how a circuit changes when priority is introduced.

5.2 Guided practice

1. Define the interface of a one-bit comparator; identify inputs and outputs in Fig. 5.2.
2. Implement the comparator with NOT, AND, and NOR gates; follow the internal structure in Fig. 5.3.
3. Create a test module that enumerates all input combinations; compare your approach with Fig. 5.4.
4. Use two one-bit comparators to build a two-bit comparator; take the hierarchy in Fig. 5.5 as a reference.

5.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

Theory block	Related exercises	Figures to consult
Module interface	Exercise 1	Fig. 5.1, Fig. 5.2
Comparator internal circuit	Exercise 2	Fig. 5.3
Test module	Exercise 3	Fig. 5.4
Hierarchy and encoders	Exercise 4 and extensions	Fig. 5.5, Fig. 5.6, Fig. 5.7

5.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 5.1 is the reference for: general diagram of a module with ports.

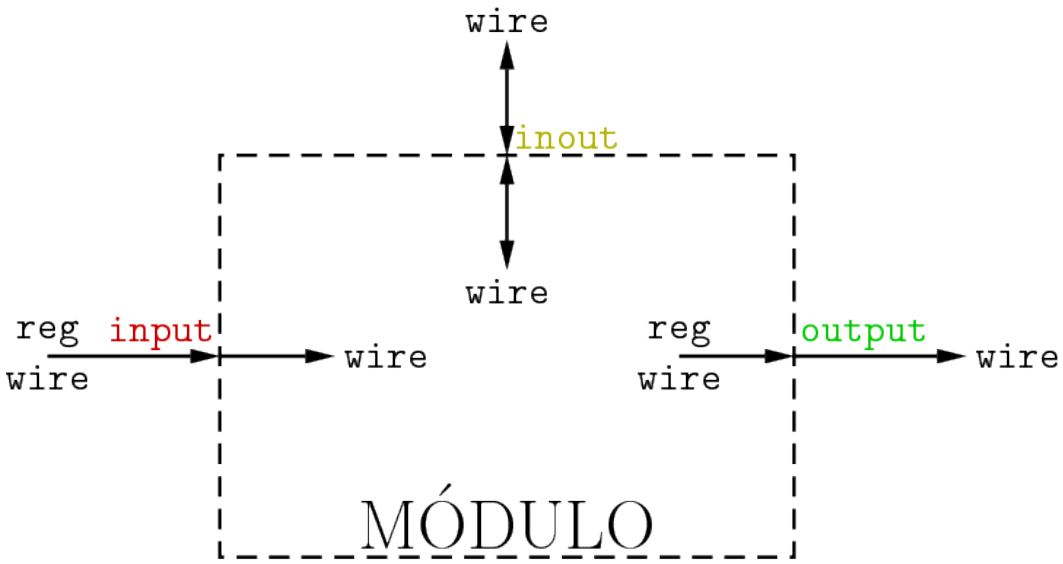


Figure5.1: General diagram of a module with ports.

The Fig. 5.2 is the reference for: one-bit comparator.

The Fig. 5.3 is the reference for: internal implementation of the one-bit comparator.

The Fig. 5.4 is the reference for: comparator test module.

The Fig. 5.5 is the reference for: two-bit comparator built hierarchically.

The Fig. 5.6 is the reference for: priority encoder.

The Fig. 5.7 is the reference for: four-input AND gate.

a	b	a > b	a = b	a < b
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

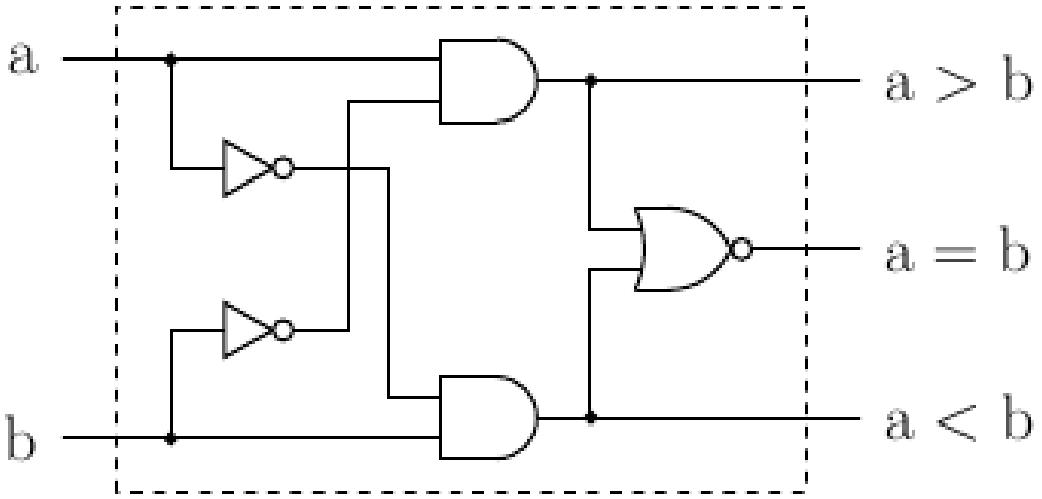


Figure5.2: One-bit comparator.

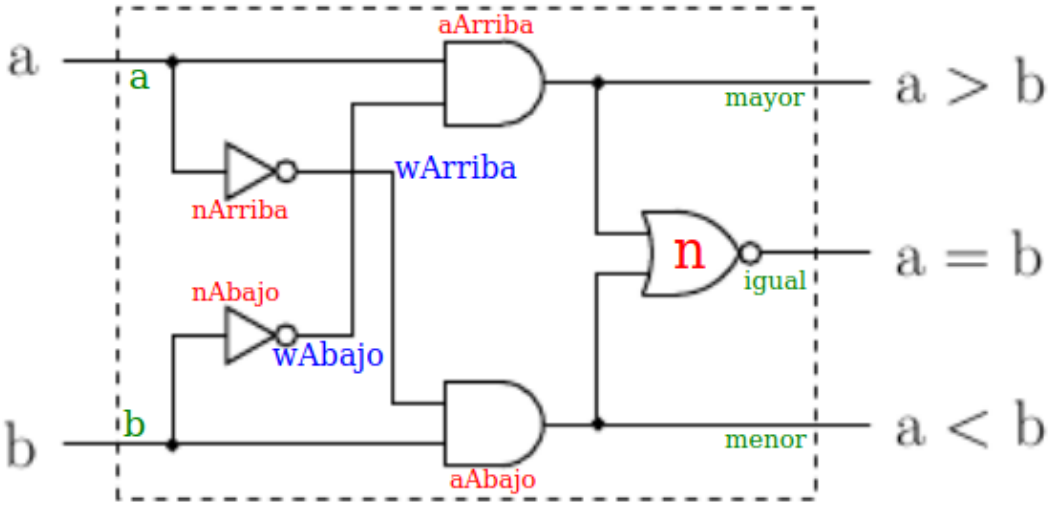


Figure5.3: Internal implementation of the one-bit comparator.

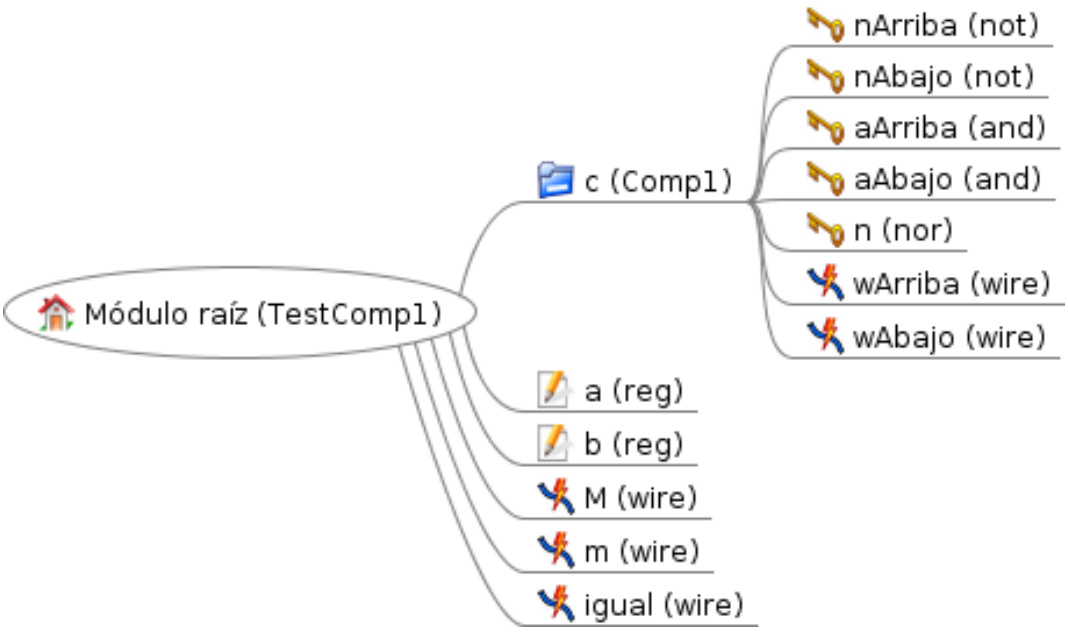


Figure5.4: Comparator test module.

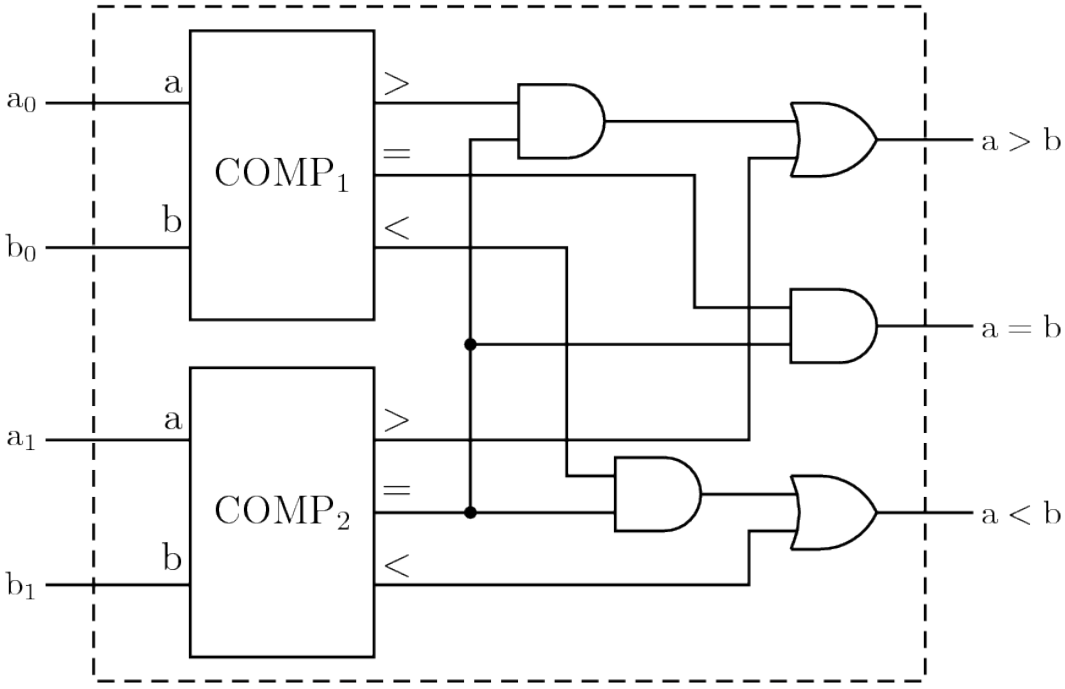


Figure5.5: Two-bit comparator built hierarchically.

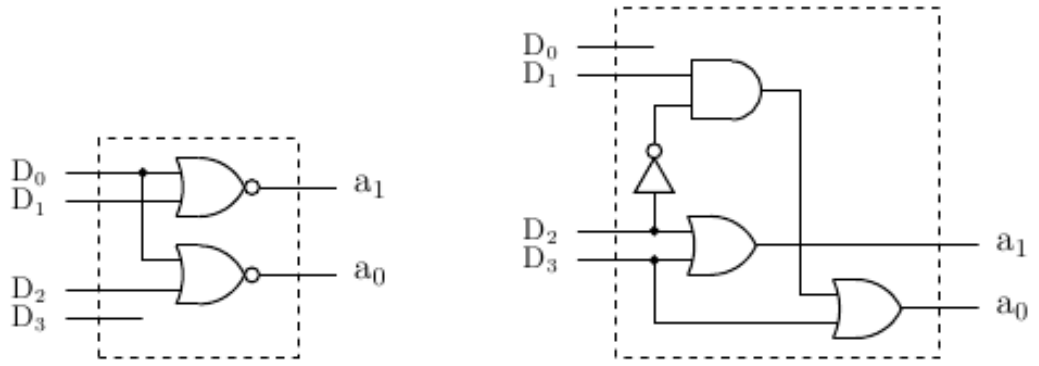


Figure5.6: Priority encoder.

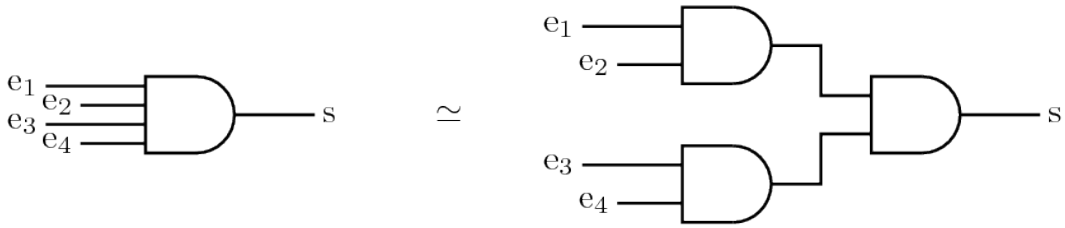


Figure5.7: Four-input AND gate.

5.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

5.6 Original source

Content translated and expanded from the class presentation and the reference page: http://avellano.fis.usal.es/~compi/sesion4.htm.

CHAPTER 6

SESSION 5. ROUTERS AND ADDERS

This session works with buses, tri-state buffers, multiplexers, and adders. The thread that connects them is controlling where a signal flows and how carry propagates.

Learning objectives

- Tri-state buffers can place an output in high impedance.
- When several signals reach the same wire, types such as `tri`, `tri0`, `tri1`, `wand`, or `wor` are useful.
- `assign` creates a continuous connection between an expression and a wire.

6.1 Key concepts

- Tri-state buffers can place an output in high impedance.
- When several signals reach the same wire, types such as `tri`, `tri0`, `tri1`, `wand`, or `wor` are useful.
- `assign` creates a continuous connection between an expression and a wire.
- Multiplexers select one input among several using control lines.
- A full adder can be built from half adders and auxiliary gates.

6.2 Guided practice

1. Program a one-bit bus transceiver; use Fig. 6.3 and Fig. 6.4 to name the signals.
2. Build a 4x1 multiplexer with output enable and then an 8x1 multiplexer; use Fig. 6.6 as a reference.
3. Implement a half adder and a full adder; relate your code to Fig. 6.8 and Fig. 6.9.
4. Add delays and estimate the safe stabilization time in a four-bit adder; justify the calculation with Fig. 6.10 and compare it with Fig. 6.11.

6.3 Theory-practice-figures index

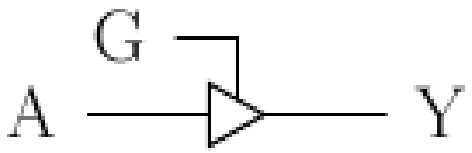
Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

Theory block	Related exercises	Figures to consult
Tri-state and buses	Exercise 1	Fig. 6.1, Fig. 6.2, Fig. 6.3, Fig. 6.4, Fig. 6.5
Multiplexers	Exercise 2	Fig. 6.6, Fig. 6.7
Adders	Exercise 3	Fig. 6.8, Fig. 6.9
Delays and carry	Exercise 4	Fig. 6.10, Fig. 6.11

6.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

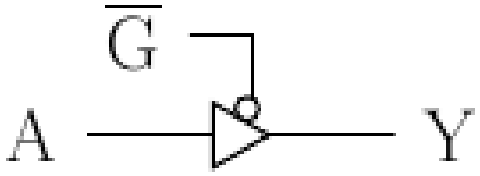
The Fig. 6.1 is the reference for: tri-state buffer active high.



G	A	Y
0	x	z
1	0	0
1	1	1

Figure6.1: Tri-state buffer active high.

The Fig. 6.2 is the reference for: tri-state buffer active low.



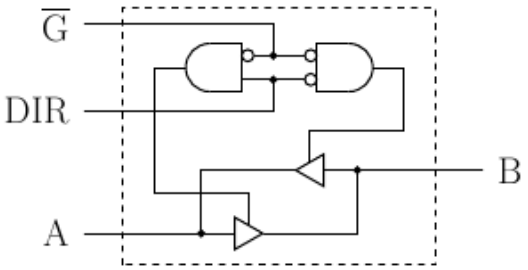
$\overline{G}$	A	Y
0	0	0
0	1	1
1	x	z

Figure6.2: Tri-state buffer active low.

The Fig. 6.3 is the reference for: one-bit bus transceiver.

The Fig. 6.4 is the reference for: auxiliary signal names in the transceiver.

The Fig. 6.5 is the reference for: transceiver test module.



$\overline{G}$	DIR	
0	0	datos B $\rightarrow$ A
0	1	datos A $\rightarrow$ B
1	x	separación

Figure6.3: One-bit bus transceiver.

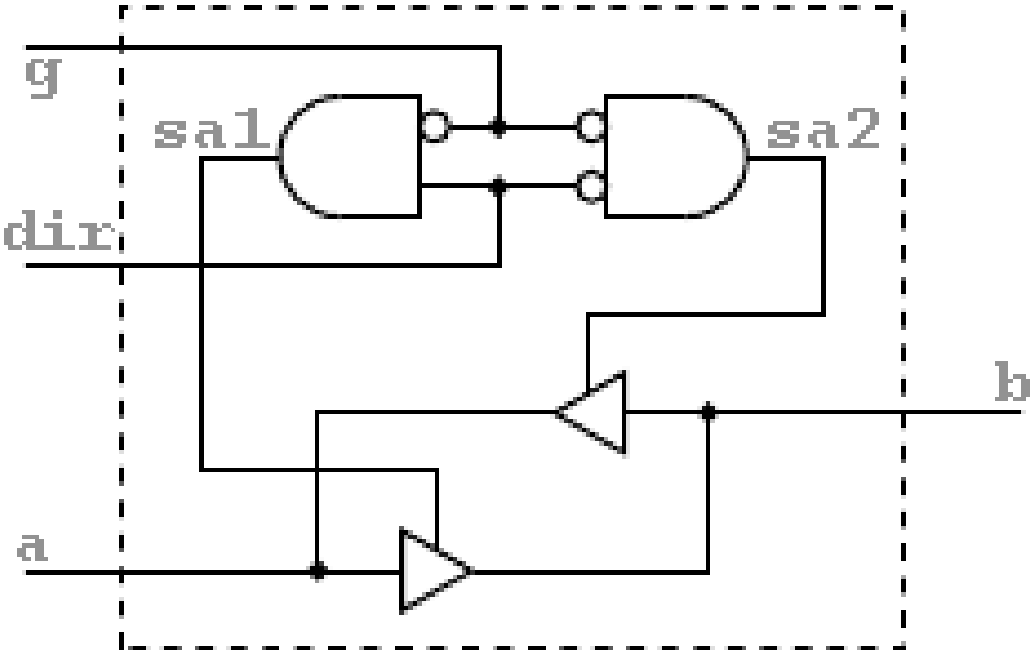


Figure6.4: Auxiliary signal names in the transceiver.

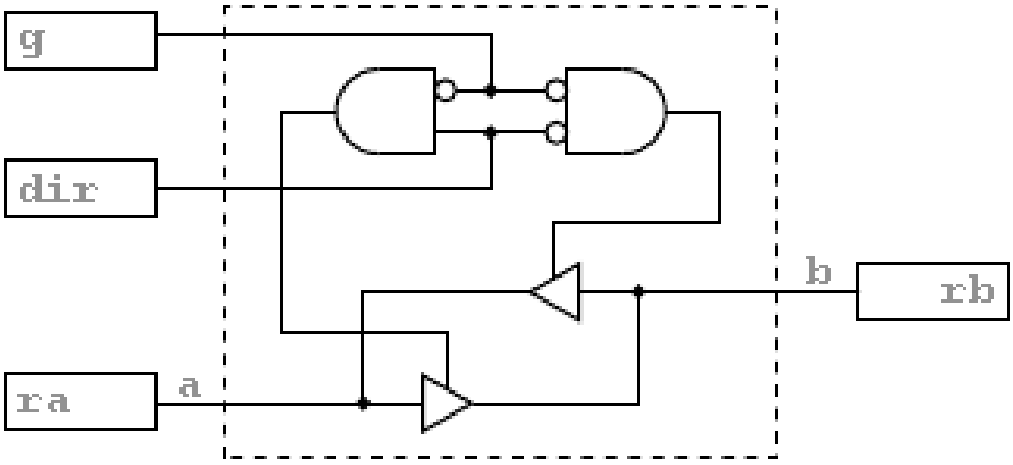


Figure6.5: Transceiver test module.

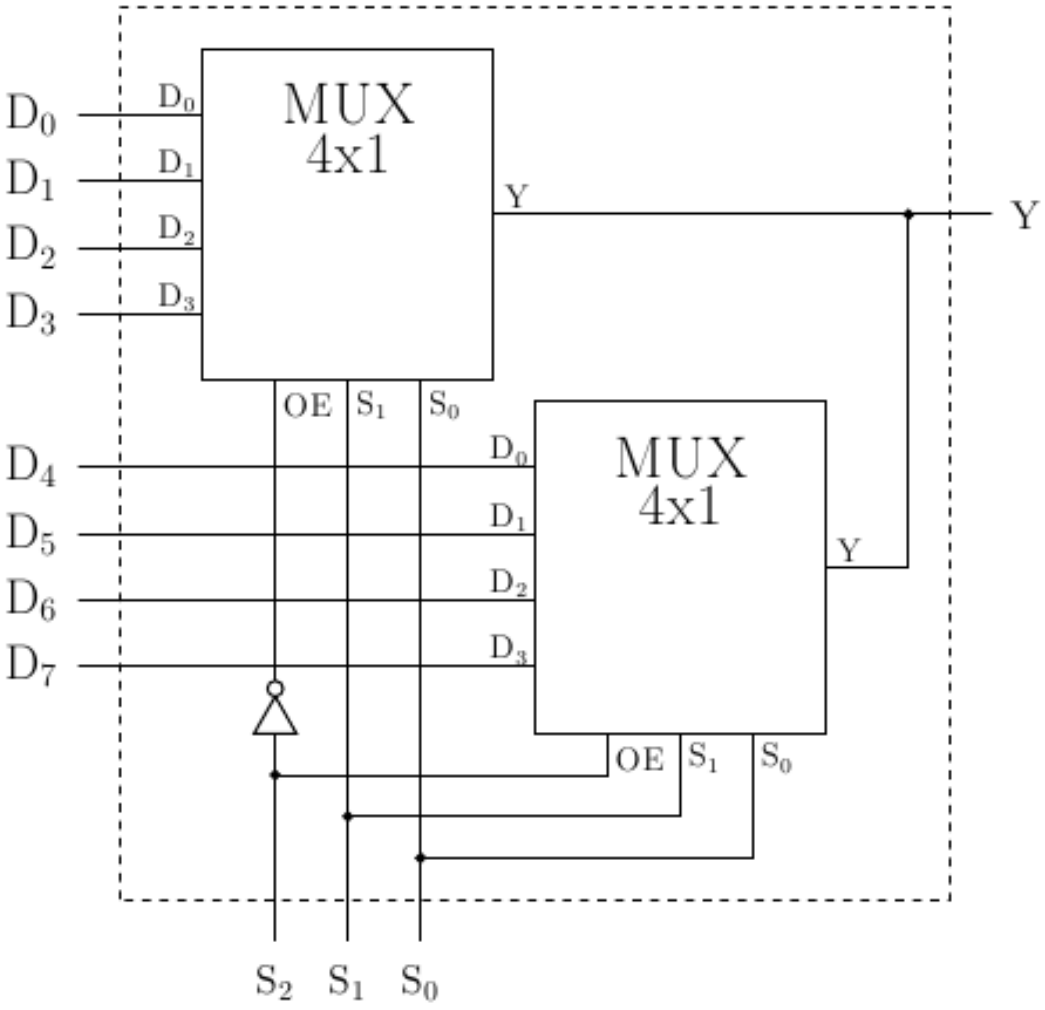


Figure6.6: 8x1 multiplexer built from smaller multiplexers.

The Fig. 6.6 is the reference for: 8x1 multiplexer built from smaller multiplexers.

The Fig. 6.7 is the reference for: auxiliary structure for signal selection.

$$h(a,b,c,d) = \overline{a}\overline{b}\,\overline{d} + b\overline{c}d + ab + a\overline{c}$$

Figure6.7: Auxiliary structure for signal selection.

The Fig. 6.8 is the reference for: half adder.

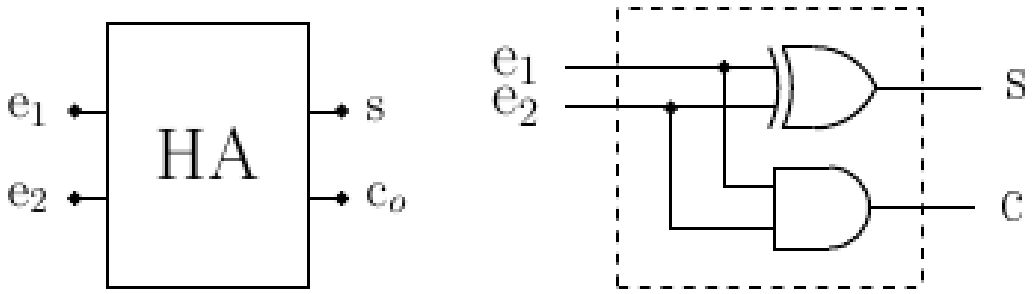


Figure6.8: Half adder.

The Fig. 6.9 is the reference for: one-bit full adder.

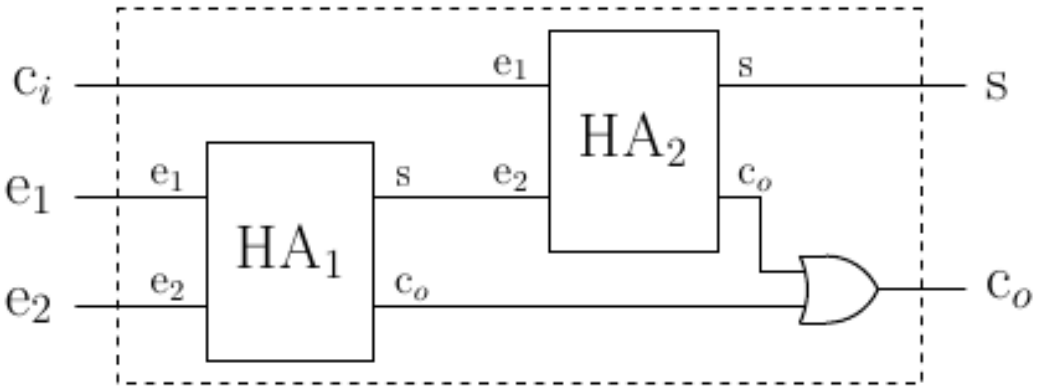


Figure6.9: One-bit full adder.

The Fig. 6.10 is the reference for: four-bit ripple-carry adder.

The Fig. 6.11 is the reference for: carry-lookahead adder.

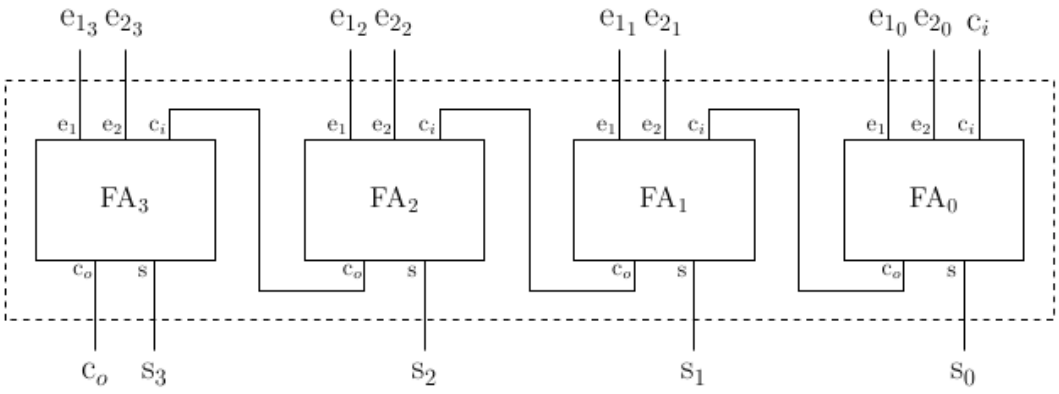


Figure6.10: Four-bit ripple-carry adder.

6.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

6.6 Original source

Content translated and expanded from the class presentation and the reference page: http://avellano.fis.usal.es/~compi/sesion5.htm.

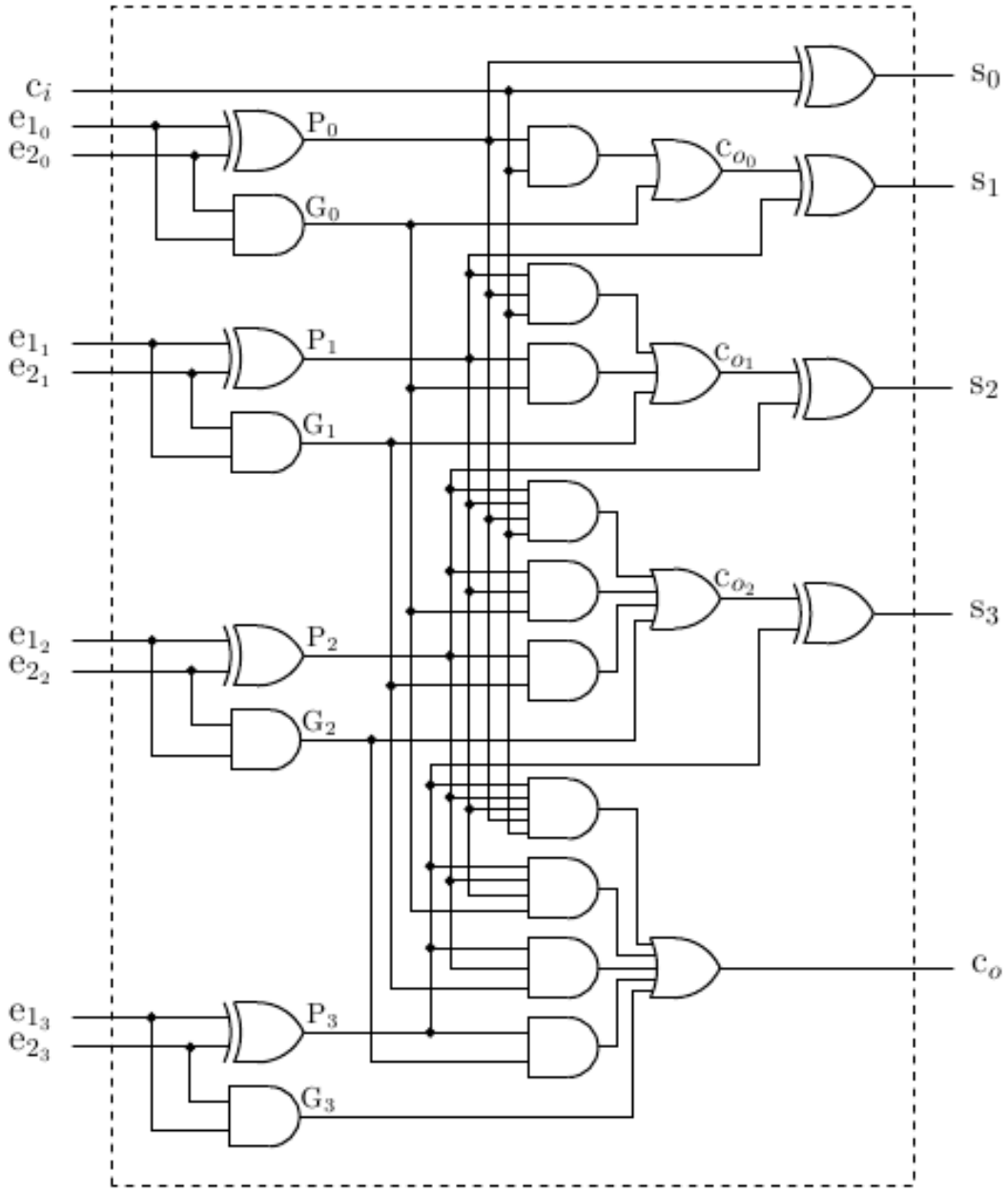


Figure6.11: Carry-lookahead adder.

CHAPTER 7

SESSION 6. FLIP-FLOPS

This session moves from combinational logic to memory. Flip-flops keep state, which forces us to reason about clocks, edges, asynchronous inputs, and delays.

Learning objectives

- An RS flip-flop can be built with two cross-coupled NOR gates.
- Blocking and non-blocking assignments do not always have the same timing effect.
- A clock can be generated with an `always` block that periodically toggles a signal.

7.1 Key concepts

- An RS flip-flop can be built with two cross-coupled NOR gates.
- Blocking and non-blocking assignments do not always have the same timing effect.
- A clock can be generated with an `always` block that periodically toggles a signal.
- Edge detectors convert a level condition into a short pulse.
- PRESET and CLEAR are often asynchronous and take priority over the clock.

7.2 Guided practice

1. Program an RS flip-flop and run the proposed input sequence; consult Fig. 7.1 and compare the results with Fig. 7.2.
2. Replace some assignments with non-blocking assignments and observe the differences near the invalid states shown in Fig. 7.2.
3. Add a level-sensitive clock input; use Fig. 7.3 and Fig. 7.4 to check when the output should change.
4. Build an edge-triggered version and then a JK flip-flop with PRESET and CLEAR; follow Fig. 7.5, Fig. 7.6, and Fig. 7.7.

7.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

Theory block	Related exercises	Figures to consult
RS flip-flop	Exercises 1 and 2	Fig. 7.1, Fig. 7.2
Level-sensitive clock	Exercise 3	Fig. 7.3, Fig. 7.4
Edges and JK	Exercise 4	Fig. 7.5, Fig. 7.6, Fig. 7.7

7.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 7.1 is the reference for: rS flip-flop with NOR gates.

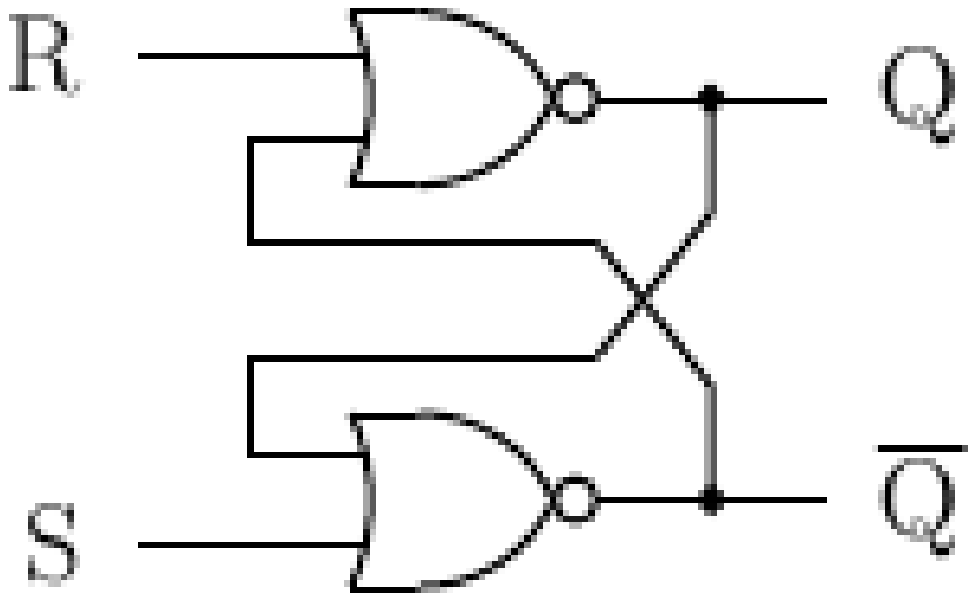


Figure7.1: RS flip-flop with NOR gates.

The Fig. 7.2 is the reference for: rS flip-flop behavior table.

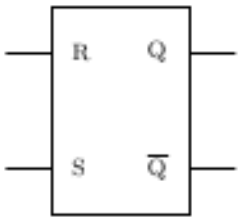
The Fig. 7.3 is the reference for: clocked RS flip-flop.

The Fig. 7.4 is the reference for: clocked RS flip-flop behavior table.

The Fig. 7.5 is the reference for: edge detector.

The Fig. 7.6 is the reference for: jK flip-flop with control inputs.

The Fig. 7.7 is the reference for: jK flip-flop behavior table.



Función	R	S	Q_{n+1}	$\overline{Q}_{n+1}$
Neutro	0	0	Q_n	$\overline{Q}_n$
Set	0	1	1	0
Reset	1	0	0	1
Inválido	1	1	?	?

Figure7.2: RS flip-flop behavior table.

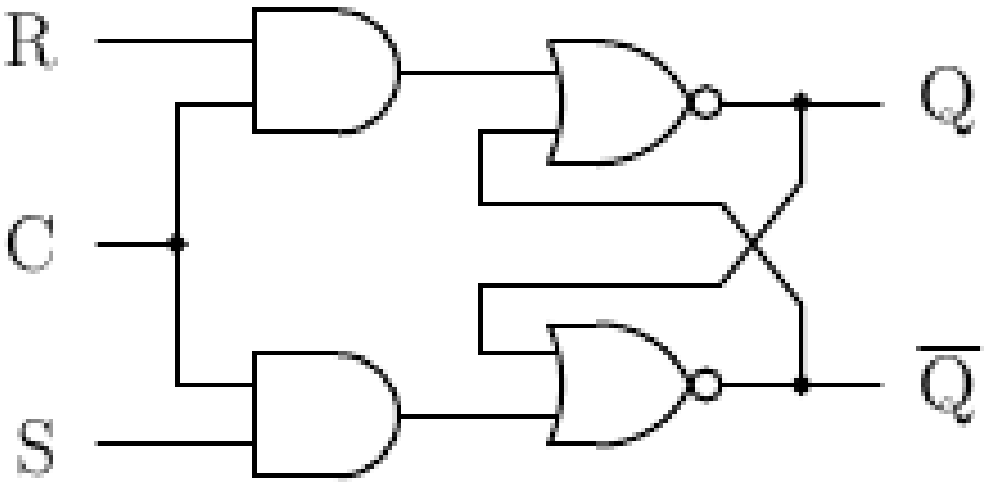
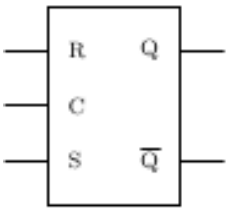


Figure7.3: Clocked RS flip-flop.



Función	C	R	S	Q_{n+1}	$\overline{Q}_{n+1}$
Neutro	0	x	x	Q_n	$\overline{Q}_n$
Neutro	1	0	0	Q_n	$\overline{Q}_n$
Set	1	0	1	1	0
Reset	1	1	0	0	1
Inválido	1	1	1	?	?

Figure7.4: Clocked RS flip-flop behavior table.

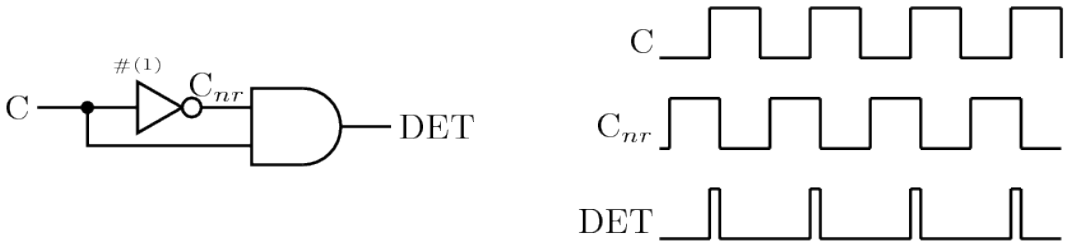


Figure7.5: Edge detector.

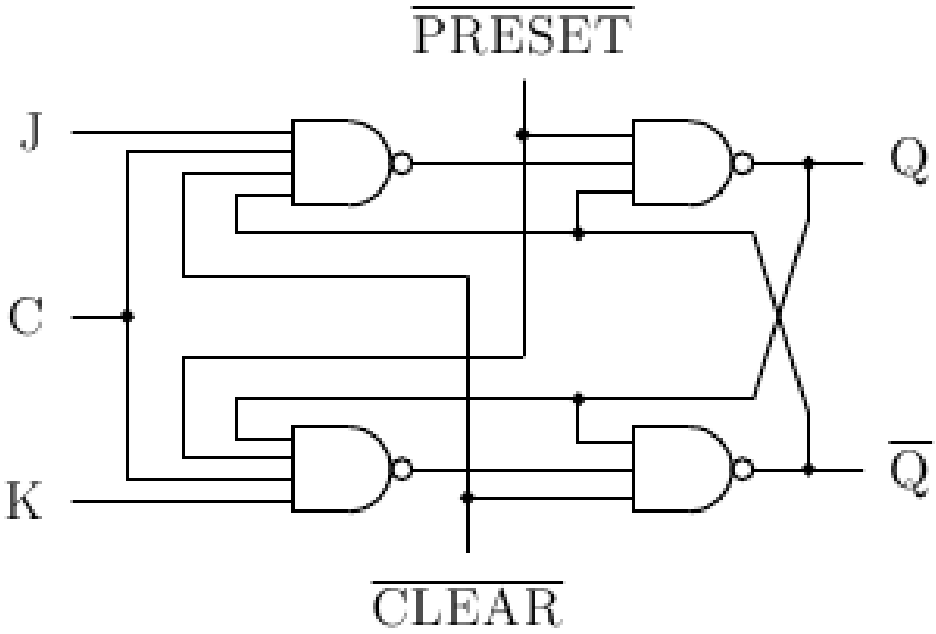
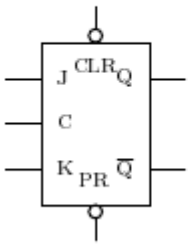


Figure7.6: JK flip-flop with control inputs.



Función	$\overline{PR}$	$\overline{CL}$	C	J	K	Q_{n+1}	$\overline{Q}_{n+1}$
Preset	0	1	x	x	x	1	0
Clear	1	0	x	x	x	0	1
Inválido	0	0	x	x	x	?	?
Neutro	1	1	1	0	0	Q_n	$\overline{Q}_n$
Set	1	1	1	1	0	1	0
Reset	1	1	1	0	1	0	1
Complemento	1	1	1	1	1	$\overline{Q}_n$	Q_n
Neutro	1	1	0	x	x	Q_n	$\overline{Q}_n$

Figure7.7: JK flip-flop behavior table.

7.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

7.6 Original source

Content translated and expanded from the class presentation and the reference page: http://avellano.fis.usal.es/~compi/sesion6.htm.

CHAPTER 8

SESSION 7. REGISTERS

This session builds registers from flip-flops and introduces an essential debugging tool: waveforms with GTKWave.

Learning objectives

- A register stores several bits distributed across flip-flops.
- SISO registers shift information serial-in, serial-out.
- SIPO registers receive data serially and output it in parallel.

8.1 Key concepts

- A register stores several bits distributed across flip-flops.
- SISO registers shift information serial-in, serial-out.
- SIPO registers receive data serially and output it in parallel.
- `$dumpfile` and `$dumpvars` generate waveform files for signal inspection.
- Real or simulated delays can explain unexpected behavior.

8.2 Guided practice

1. Build a D flip-flop from the blocks already studied; use Fig. 8.1 to verify inputs and output.
2. Use several flip-flops to form a SISO register; follow the connection in Fig. 8.2.
3. Generate a waveform file and open it with GTKWave; compare the expected window with Fig. 8.3.
4. Modify the register to obtain a parallel SIPO output; use Fig. 8.4 and, for the extension, Fig. 8.5.

8.3 Theory-practice-figures index

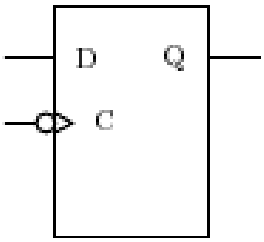
Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

Theory block	Related exercises	Figures to consult
D flip-flop	Exercise 1	Fig. 8.1
SISO register	Exercise 2	Fig. 8.2
Waveforms	Exercise 3	Fig. 8.3
SIPO registers and parallel load	Exercise 4	Fig. 8.4, Fig. 8.5

8.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 8.1 is the reference for: d flip-flop.



Función	C	Q_{n+1}
Captura	↓	D
Neutro	otro	Q_n

Figure8.1: D flip-flop.

The Fig. 8.2 is the reference for: sISO register.

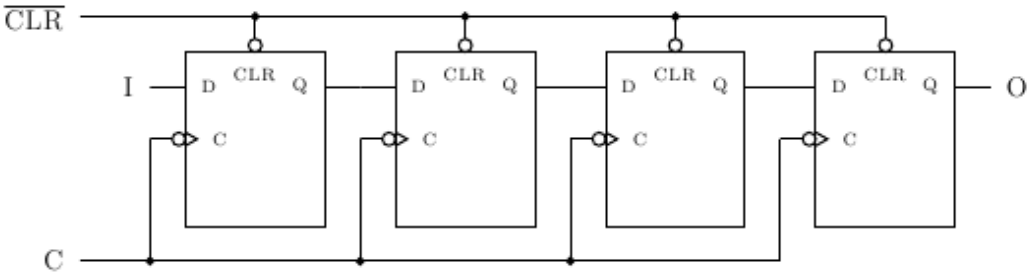


Figure8.2: SISO register.

The Fig. 8.3 is the reference for: signal visualization with GTKWave.

The Fig. 8.4 is the reference for: sIPO register.

The Fig. 8.5 is the reference for: register with parallel load and serial shift.

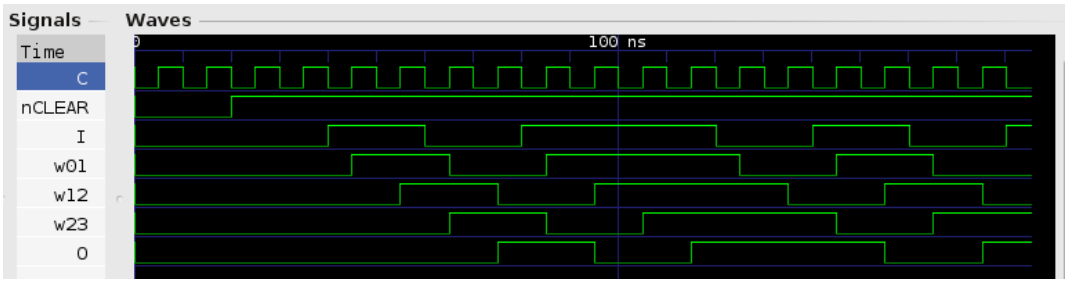


Figure8.3: Signal visualization with GTKWave.

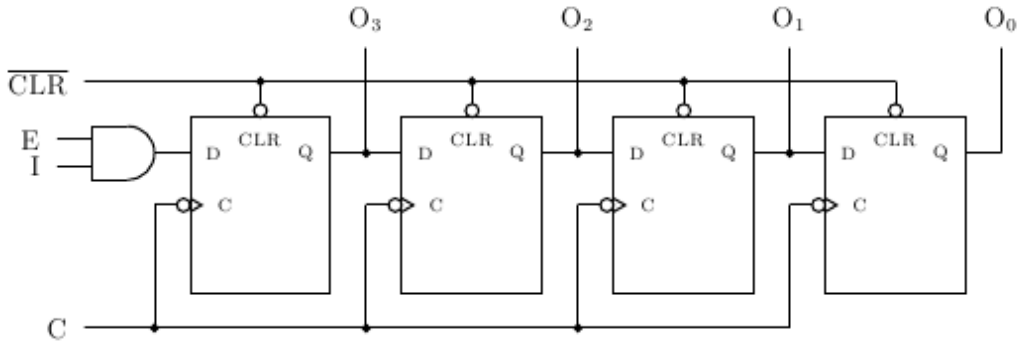


Figure8.4: SIPO register.

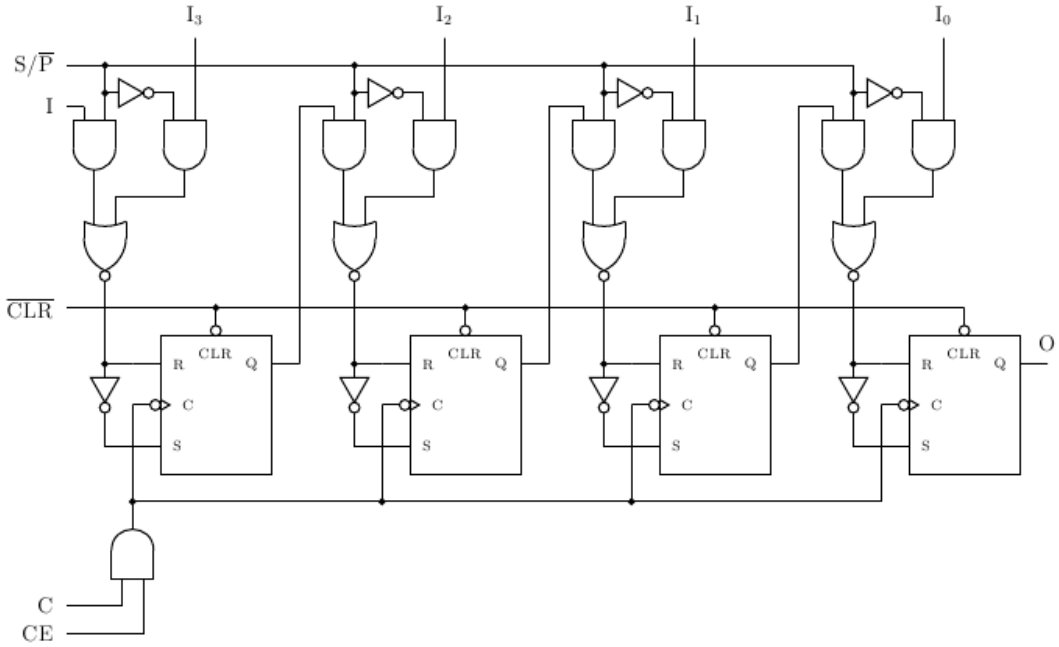


Figure8.5: Register with parallel load and serial shift.

8.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

8.6 Original source

Content translated and expanded from the class presentation and the reference page: http://avellano.fis.usal.es/~compi/sesion7.htm.

CHAPTER 9

SESSION 8. COUNTERS

This session uses flip-flops to build counters and analyze state transitions. The important point is to distinguish ideal behavior, delays, and transient states.

Learning objectives

- A counter changes state according to a clock signal.
- Counting can be upward or downward depending on a mode line.
- HOLD freezes the state when the count direction must change without jumps.

9.1 Key concepts

- A counter changes state according to a clock signal.
- Counting can be upward or downward depending on a mode line.
- HOLD freezes the state when the count direction must change without jumps.
- Asynchronous counters can pass through transient states while signals propagate.
- State transition diagrams summarize the sequential behavior of a circuit.

9.2 Guided practice

1. Program a four-bit up/down counter; use Fig. 9.1 to define inputs, outputs, and count mode.
2. Observe what happens when changing mode without clearing the counter; explain the transient states with Fig. 9.4.
3. Add PRESET, CLEAR, and HOLD to control transitions better; compare your design with Fig. 9.3 and Fig. 9.5.
4. Analyze an arbitrary-count counter and draw its state diagram; use Fig. 9.7, Fig. 9.8, and Fig. 9.9.

9.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

Theory block	Related exercises	Figures to consult
Up/down counting	Exercises 1 and 2	Fig. 9.1, Fig. 9.4
Transition control	Exercise 3	Fig. 9.2, Fig. 9.3, Fig. 9.5
States and JK flip-flops	Exercise 4	Fig. 9.6, Fig. 9.7, Fig. 9.8, Fig. 9.9

9.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 9.1 is the reference for: counter with up/down selection.

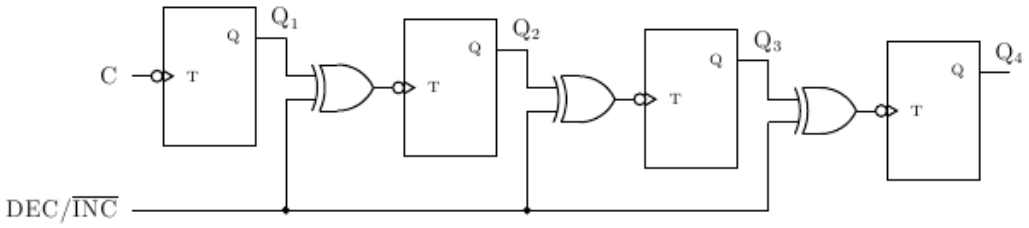


Figure9.1: Counter with up/down selection.

The Fig. 9.2 is the reference for: counter from 10 down to 0.

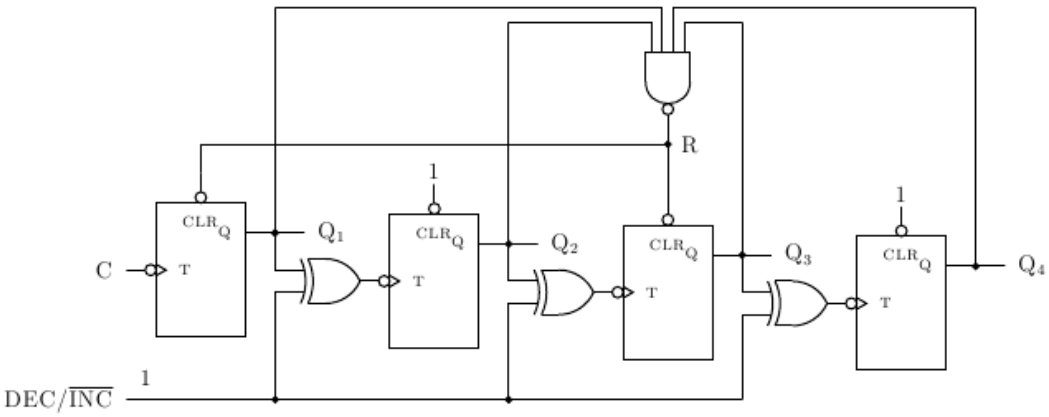


Figure9.2: Counter from 10 down to 0.

The Fig. 9.3 is the reference for: synchronous counter.

The Fig. 9.4 is the reference for: analysis of states and transitions.

The Fig. 9.5 is the reference for: auxiliary counter circuit.

The Fig. 9.6 is the reference for: transitions of a JK flip-flop.

The Fig. 9.7 is the reference for: state transition diagram.

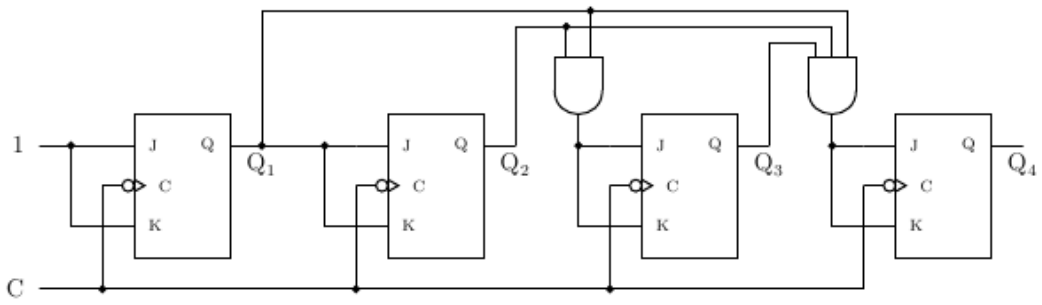


Figure9.3: Synchronous counter.

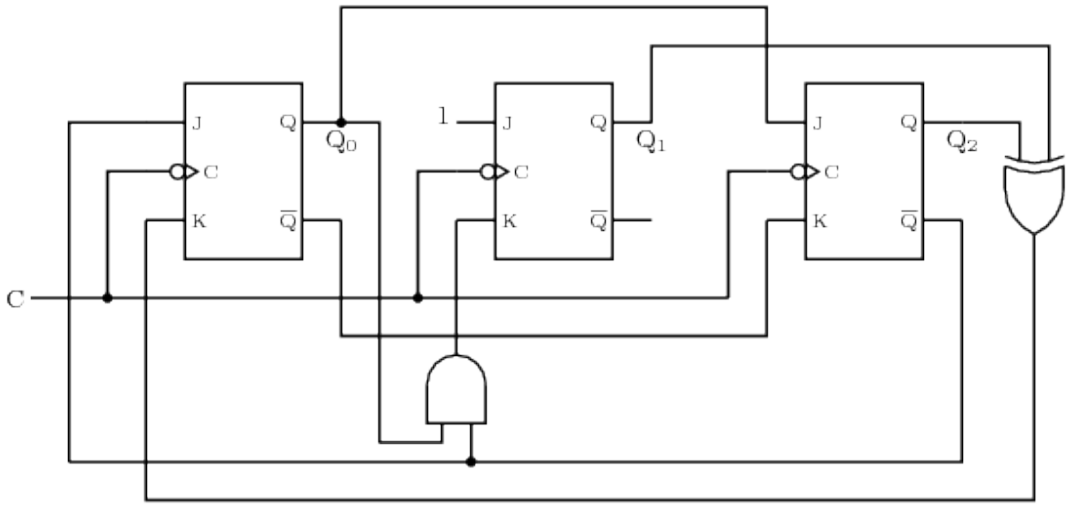


Figure9.4: Analysis of states and transitions.

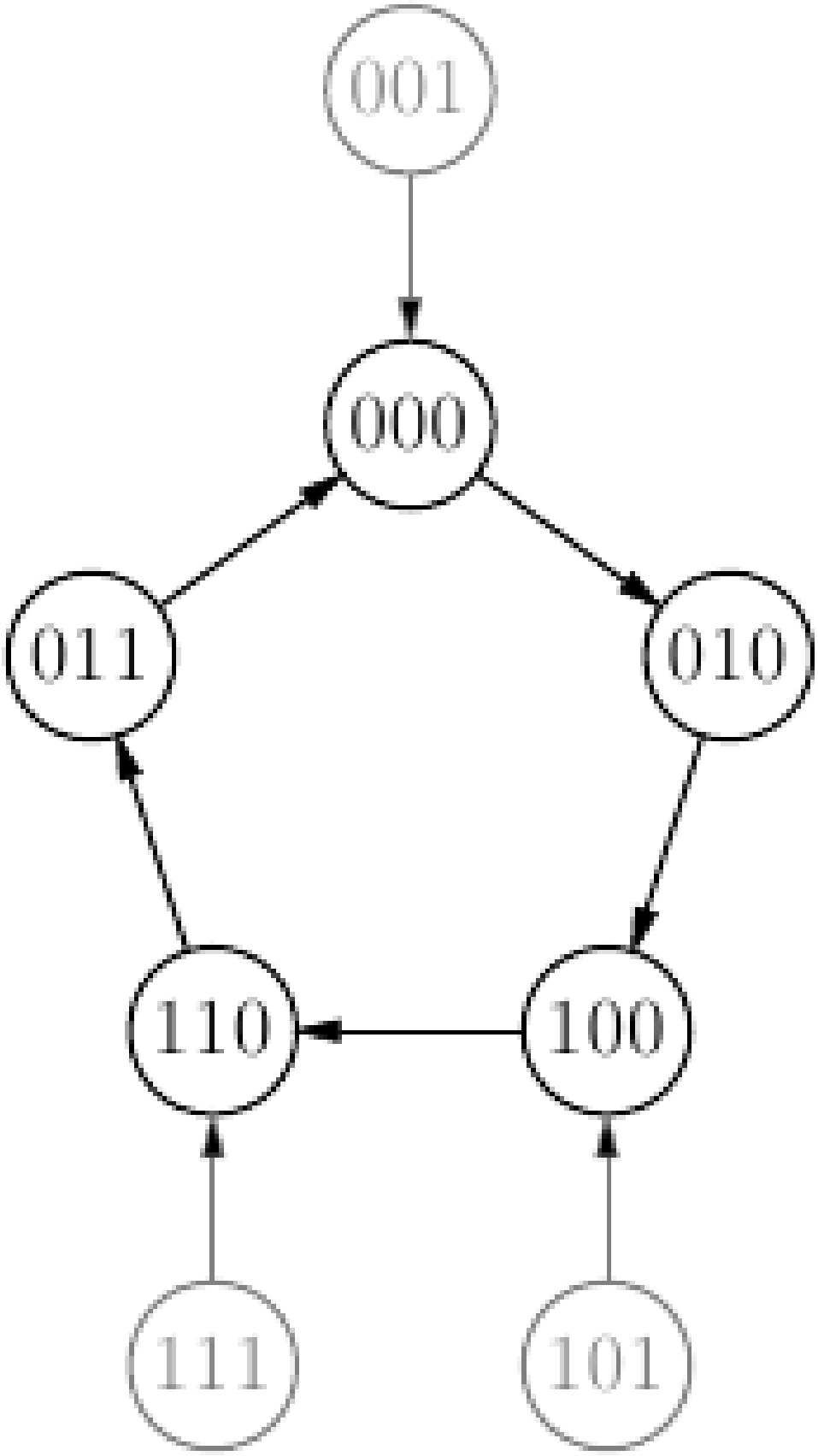


Figure9.5: Auxiliary counter circuit.

Actual	Sgte.	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Figure9.6: Transitions of a JK flip-flop.

Transición	J_2	K_2	J_1	K_1	J_0	K_0
$000 \rightarrow 010$	0	x	1	x	0	x
$001 \rightarrow 000$	0	x	0	x	x	1
$010 \rightarrow 100$	1	x	x	1	0	x
$011 \rightarrow 000$	0	x	x	1	x	1
$100 \rightarrow 110$	x	0	1	x	0	x
$101 \rightarrow 100$	x	0	0	x	x	1
$110 \rightarrow 011$	x	1	x	0	1	x
$111 \rightarrow 110$	x	0	x	0	x	1

Figure9.7: State transition diagram.

The Fig. 9.8 is the reference for: karnaugh map for JK inputs.

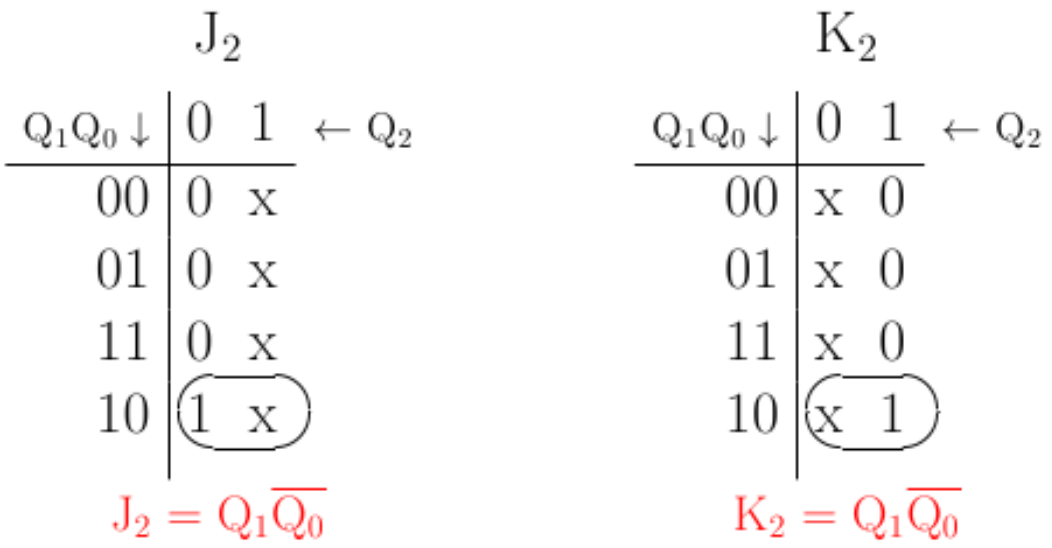


Figure9.8: Karnaugh map for JK inputs.

The Fig. 9.9 is the reference for: arbitrary-count counter.

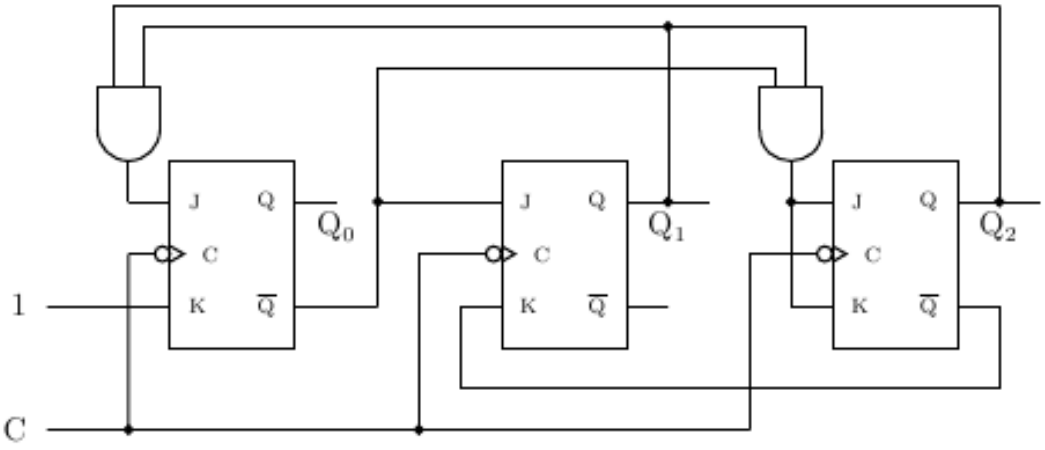


Figure9.9: Arbitrary-count counter.

9.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

9.6 Original source

Content translated and expanded from the class presentation and the reference page: http://avellano.fis.usal.es/~compi/sesion8.htm.

CHAPTER 10

SESSION 9. ALU

The final session applies modular work to a 74181 ALU. It practices file inclusion and the combination of arithmetic and logic operations.

Learning objectives

- `include` inserts the contents of another Verilog file at the indicated point.
- The 74181 ALU receives selection lines that determine the operation.
- Negated carry lines require care when interpreting inputs and outputs.

10.1 Key concepts

- `include` inserts the contents of another Verilog file at the indicated point.
- The 74181 ALU receives selection lines that determine the operation.
- Negated carry lines require care when interpreting inputs and outputs.
- Compound operations are solved by storing intermediate results.
- The test module must clearly state the inputs, selected operation, and expected output.

10.2 Guided practice

1. Include the `74181.v` file from a test module; identify the ports in Fig. 10.1.
2. Check operations such as `7+4`, `2+6+1`, AND, XOR, and OR combined with NOT/XNOR; select the control lines using Fig. 10.2.
3. Implement small multiplications as repeated additions; use Fig. 10.2 again to justify each intermediate step.
4. Document each control-line selection so that errors can be debugged; your table must explicitly refer to Fig. 10.1 and Fig. 10.2.

10.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

Theory block	Related exercises	Figures to consult
ALU interface	Exercise 1	Fig. 10.1
Operation selection	Exercises 2 to 4	Fig. 10.2

10.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 10.1 is the reference for: functional diagram of the 74181 ALU.

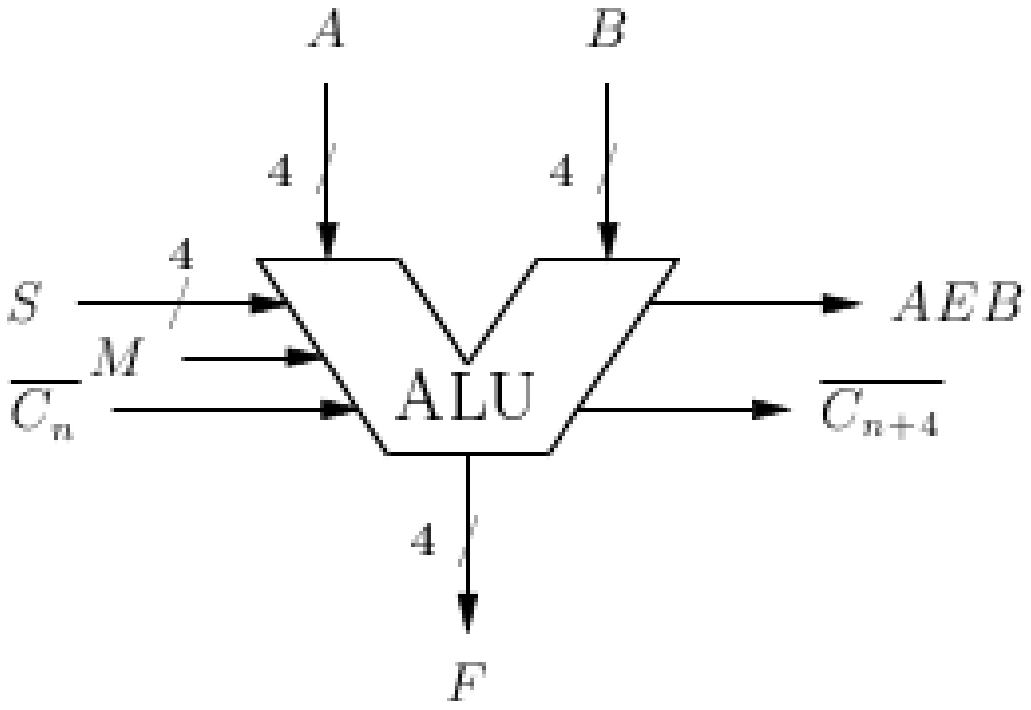


Figure10.1: Functional diagram of the 74181 ALU.

The Fig. 10.2 is the reference for: operation table of the 74181 ALU.

S	Aritmético $(M = 0)$	Lógico $(M = 1)$
0000	$A + C_n$	$\overline{A}$
0001	$A \mid B + C_n$	$\overline{A \mid B}$
0010	$A \mid \overline{B} + C_n$	$\overline{A}B$
0011	$-1 + C_n$	0
0100	$A + A\overline{B} + C_n$	$\overline{AB}$
0101	$(A \mid B) + A\overline{B} + C_n$	$\overline{B}$
0110	$A - B - 1 + C_n$	$A \oplus B$
0111	$A\overline{B} - 1 + C_n$	$A\overline{B}$
1000	$A + AB + C_n$	$\overline{A} \mid B$
1001	$A + B + C_n$	$\overline{A \oplus B}$
1010	$(A \mid \overline{B}) + AB + C_n$	B
1011	$AB - 1 + C_n$	AB
1100	$A + A + C_n$	1
1101	$(A \mid B) + A + C_n$	$A \mid \overline{B}$
1110	$(A \mid \overline{B}) + A + C_n$	$A \mid B$
1111	$A - 1 + C_n$	A

Figure10.2: Operation table of the 74181 ALU.

10.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

10.6 Original source

Content translated and expanded from the class presentation and the reference page: http://avellano.fis.usal.es/~compi/sesion9.htm.